

FedCrime

Zero-Inflation Adaptive Federated Learning for Crime Prediction

A complete study guide in simple English

Every formula (Eq. 3 to Eq. 12) explained: what it says, why it is used, what it does, and the exact Python code that implements it. Plus the data pipeline, the nine baseline methods, and how our results compare with the research paper.

Based on: Bhumika, Lalanda, Vega, Das. *FedCrime*, Neurocomputing 679 (2026) 133217.

1. What is FedCrime, in one page

The goal. Predict, for each area of a city and for each day, which types of crime are likely to happen (theft, burglary, assault, and so on). This is called *spatio-temporal crime prediction* - *spatio* means space (city areas) and *temporal* means time (days).

The privacy problem. Older methods collect all the crime data from every police precinct into one central computer and train one model there. Crime data is sensitive, so gathering it in one place is a privacy and security risk.

The federated idea. *Federated Learning (FL)* keeps each area's data on that area's own machine (called a *client*). Each client trains a small model on its own data and only sends the trained *numbers* (model weights) to a central server - never the raw data. The server averages all the clients' weights into one shared *global model* and sends it back. This repeats many times (each repeat is one *communication round*).

The hard problem this paper solves. Crime is very unevenly spread. A few busy areas have lots of crime (the *head*), but most areas have almost no crime (the *tail*). So most days in most areas the answer is simply "no crime" - a value of zero. So many zeros is called **zero-inflation**. Ordinary models get confused by all these zeros: the quiet areas learn nothing useful, and when their weak models are averaged in, they drag down the good areas. This is called *negative knowledge transfer*.

Key idea: FedCrime fixes zero-inflation at the **loss function** (the maths that measures error), not at the server. Because each client's learning is already made aware of sparsity, the server can just do simple averaging (FedAvg) and it still works well.

Two building blocks:

1. **TCN-SD model** (on each client) - a *Temporal Convolutional Network* (TCN) that reads the recent history of days, joined to a *Statistical Distribution* (SD) head that outputs the pieces of a special probability distribution suited to counting rare events.
2. **ZINB loss** - a *Zero-Inflated Negative Binomial* error function that separately handles "why is this a zero?" and "how big is this non-zero count?". This is the heart of the paper.

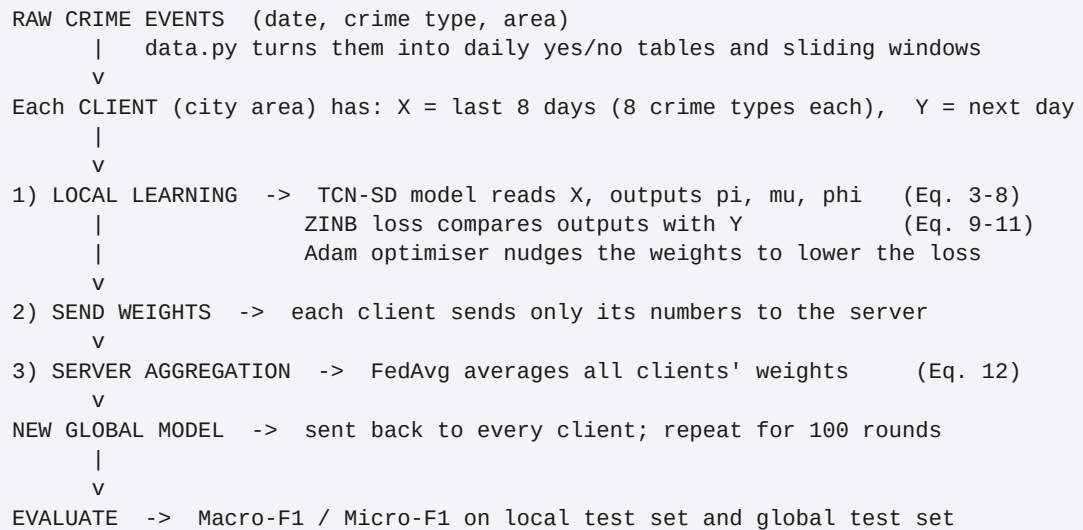
2. Words you need to know (simple definitions)

Term	Simple meaning
Client / region	One city area with its own private data and its own local copy of the model.
Communication round	One cycle of: clients train locally, send weights, server averages, sends back.
Weights	The learnable numbers inside the model. Training just adjusts these numbers.
Zero-inflation	Having far more zeros (no-crime days) than a normal model expects.
Long-tail	A few areas with lots of crime, and a long 'tail' of many areas with almost none.
Over-dispersion	When the counts vary much more than a simple model would predict.
TCN	Temporal Convolutional Network: reads a sequence (past days) using sliding filters.
Dilation	Skipping steps in the filter so it can 'see' far back in time cheaply.
Causal	The model at day t only uses days up to t - it cannot peek into the future.

Negative Binomial (NB)	A probability distribution good at modelling counts that are spread out.
ZINB	Zero-Inflated NB: an NB plus an extra switch that explains the extra zeros.
Multi-label	Each day can have several crime types at once, each marked 1 (yes) or 0 (no).
Macro-F1	Average score treating every crime type equally - rewards doing well on rare types.
Micro-F1	Overall score dominated by the common crime types (like theft).
FedAvg	Federated Averaging: the server just averages the clients' weights.

3. How the whole system works (the flow)

Follow the journey of one training round:



Key idea: Numbers 1, 2, 3 above map exactly to the three arrows in Figure 1 of the paper: local learning, model-update transmission, server-side aggregation.

4. The data pipeline (file: data.py)

Datasets. Los Angeles 2018 (113 areas) and Chicago 2015 (77 areas). Each has 8 crime categories. LA is the harder one because it has more zeros.

Step A - make daily yes/no tables. For each area we build a table with one row per day and one column per crime type. The cell is 1 if that crime happened that day, else 0. This is the 'multi-label binary' form.

Step B - sliding windows. We slide a window of 10 days across the calendar. The first 8 days become the input X; the next day becomes the label Y we try to predict. This gives many (X, Y) training examples.

Step C - split by time. Exactly like the paper: about 6.5 months for training, 0.5 month for validation, 1 month for testing (0.5417 / 0.0417 / 0.0833 of the year). We split by time, never randomly, so we always predict the future from the past.

Step D - Head / Mid / Tail. We sort areas by total crime and cut at the 20% / 30% / 50% marks: top 20% = Head (busy), next 30% = Mid, bottom 50% = Tail (quiet). This lets us build subsets with controlled difficulty.

Step E - the four subsets (10 clients each):

Subset	Made of	Meaning	LA sparsity
SET-Alpha	10 Head	easiest - only busy areas	~40%
SET-Beta	5 Head + 5 Mid	medium	~49%
SET-Gamma	3 Head + 4 Mid + 3 Tail	hard	~59%
SET-Omega	2 Head + 3 Mid + 5 Tail	hardest - most like a real city	~65%

The code that builds one area's windows (input X and label Y):

```
for win in _moving_window(days, WINDOW):      # WINDOW = 10
    hist_days = win[:WINDOW - 1]              # first 9 days
    x = np.stack([mat.loc[day] for day in hist_days[:-1]]) # 8 days x 8 types -> X
    y = mat.loc[win[WINDOW - 2]]              # the next day -> Y (8 yes/no labels)
```

5. The TCN-SD model (file: model.py)

The model has two parts. Part 1 (the TCN) reads the 8 days of history and squeezes them into a summary. Part 2 (the SD head) turns that summary into three numbers per crime type that describe a ZINB distribution.

Part 1 - the TCN blocks

Equation 3 and 4 - causal dilated convolution.

$$\begin{aligned}\text{Eq 3: } h(l, t) &= \text{ReLU}(W_1 \cdot X(t - d \cdot k) + b_1), \quad k = 0 \dots K-1 \\ \text{Eq 4: } h(l+1, t) &= \text{ReLU}(W_2 \cdot h(l, t - d \cdot k) + b_2)\end{aligned}$$

What it says (simple). A convolution slides a small filter (W) over the sequence of days. At day t it looks at day t and a few earlier days, multiplies them by learned weights, adds a bias b , and passes the result through ReLU (which just keeps positive values and sets negatives to 0). 'Causal' means it only looks backward in time. 'Dilated' (the d) means it skips days - with $d=2$ it looks at days $t, t-2, t-4, \dots$ so with few layers it can still reach far into the past.

Why it is used. Crime today depends on recent days (weekends, trends). The TCN captures these time patterns. Compared to older RNN/LSTM models it is faster and easier to train. Dilation gives a big memory cheaply.

What happens. Each layer produces a richer summary of the recent past. We stack 3 such layers with growing dilation (1, 2, 4).

Equation 5 - residual (shortcut) connection.

$$\text{Eq 5: } h_{\text{tcn}}(t) = h(l+1, t) + \text{Residual}(X(t - d \cdot k))$$

What it says. Add the block's input back onto its output (a shortcut). If the shapes differ, a tiny 1x1 convolution reshapes the input first.

Why. Shortcuts let gradients flow during training, so deep stacks train stably and do not 'forget' the original signal. This is the standard ResNet trick.

The code (one TCN block = Eq 3, 4, 5 together):

```
class TCNBlock(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size=3, dilation=1):
        padding = (kernel_size - 1) * dilation // 2
        self.conv1 = nn.Conv1d(in_channels, out_channels, kernel_size, padding=padding, dilation=dilation)
        self.conv2 = nn.Conv1d(out_channels, out_channels, kernel_size, padding=padding, dilation=dilation)
        self.relu = nn.ReLU()
        self.residual = nn.Conv1d(in_channels, out_channels, 1) if in_channels != out_channels else nn.Identity()
    def forward(self, x):
        res = self.residual(x)           # the shortcut (Eq 5)
        x = self.relu(self.conv1(x))     # first conv + ReLU (Eq 3)
        x = self.relu(self.conv2(x))     # second conv + ReLU (Eq 4)
        return x + res                   # add shortcut back (Eq 5)
```

Three blocks are stacked with dilation 2^i so the model sees a long history:

```
for i in range(num_layers):
    in_ch = num_inputs if i == 0 else hidden_units
    layers.append(TCNBlock(in_ch, hidden_units, kernel_size, dilation=2 ** i)) # d = 1, 2, 4
```

Part 2 - the SD head (three outputs of a ZINB)

After the TCN we take the summary of the last day, h . Three separate small linear layers turn h into three numbers per crime type. Each number is squeezed by a function that forces it into the correct range.

Equations 6, 7, 8 - the three ZINB parameters.

```
Eq 6: pi = sigmoid ( W_pi . h + b_pi )    -> zero-inflation probability (0..1)
Eq 7: mu = exp      ( W_mu . h + b_mu )    -> the mean count (always > 0)
Eq 8: phi = softplus( W_phi . h + b_phi )  -> the dispersion (always > 0)
```

What each one means (simple).

- **pi** (Eq 6): the chance that this cell is an 'extra zero' - a structural no-crime. *sigmoid* squeezes any number into 0..1, so it reads like a probability. In this project π is also used as the final 'is there crime?' score.

- **mu** (Eq 7): the average count we expect if crime does occur. *exp* (the exponential) makes sure it is always a positive number.

- **phi** (Eq 8): how spread-out / bursty the counts are (over-dispersion). *softplus* is a smooth way to force a positive number.

Why three numbers? Because a ZINB distribution needs exactly these three ingredients: a switch for extra zeros (π), a mean (μ), and a spread (ϕ). Producing them lets the model describe rare, bursty, mostly-zero crime data honestly instead of pretending it is a smooth bell curve.

The code (the SD head and the full forward pass):

```
self.fc_zero = nn.Linear(hidden_units, num_categories) # -> pi
self.fc_mean = nn.Linear(hidden_units, num_categories) # -> mu
self.fc_disp = nn.Linear(hidden_units, num_categories) # -> phi

def forward(self, x):
    x = x.permute(0, 2, 1)          # arrange as (batch, channels, days)
    h = self.tcn(x)                  # run the 3 TCN blocks
    h = h[:, :, -1]                  # keep the summary of the LAST day
    pi = self.sigmoid(self.fc_zero(h)) # Eq 6
    mu = torch.exp(torch.clamp(self.fc_mean(h), max=15.0)) # Eq 7 (clamp = overflow-safe)
    phi = self.softplus(self.fc_disp(h)) # Eq 8
    return pi, mu, phi
```

(The clamp on Eq 7 is a small safety guard we added: it stops `exp()` from ever exploding to infinity if a client's training diverges. It does not change the maths in normal use.)

6. The ZINB loss - the heart of FedCrime (file: loss.py)

The loss is a single number that says 'how wrong were the predictions?'. Training tries to make it small. FedCrime's special loss has two parts: one for the zeros, one for the non-zeros. It uses the π , μ , ϕ from the model.

Equation 9 - the zero part.

Eq 9: $L_{\text{zero}} = 1\{Y = 0\} * \log(\pi + \text{eps})$ $\text{eps} = 1e-10$

What it says. Only for cells where the true answer is zero (the $1\{Y=0\}$ is a switch that is 1 when Y is zero, else 0), reward the model for giving a high π . In words: 'when there really was no crime, the model should be confident it is an extra zero.' The tiny eps just avoids $\log(0)$.

Equation 10 - the non-zero part (Negative Binomial).

Eq 10: $L_{\text{NB}} = 1\{Y = 1\} * [$
 $\log\Gamma(Y + \phi) - \log\Gamma(\phi) - \log\Gamma(Y + 1)$
 $+ \phi * (\log \phi - \log(\phi + \mu))$
 $+ Y * (\log \mu - \log(\phi + \mu))]$

What it says. Only for cells where crime did occur, this measures how well the μ and ϕ explain that count using the Negative Binomial formula. $\log\Gamma$ is a smooth version of factorial used inside that formula. You do not need to memorise every term - the idea is: 'when crime happened, score how well the count distribution fits.'

Equation 11 - put them together.

Eq 11: $J_i = - E[L_{\text{zero}} + L_{\text{NB}}]$ (averaged over the client's samples)

What it says. Add the zero-part and non-zero-part, average over all examples, and put a minus sign in front. (These L terms are log-likelihoods - 'how likely'. We want likelihood high, so we minimise its negative.)

Key idea: Why this is clever. Quiet (tail) areas are almost all zeros, so their loss is dominated by the gentle Eq-9 term -> their weight updates are small and calm. Busy areas with real crimes get sharp Eq-10 gradients -> they contribute strongly. So the loss itself automatically turns down the noisy quiet clients and turns up the informative busy ones. That is why plain averaging on the server is enough - no fancy server tricks needed.

The code (Eq 9, 10, 11 line by line):

```
def zinb_loss(pi, mu, phi, targets):
    is_zero = targets.eq(0).float()      # switch 1{Y=0}
    is_nonzero = targets.gt(0).float()    # switch 1{Y=1}

    zero_term = is_zero * torch.log(pi + EPS)          # Eq 9

    log_factorial = torch.lgamma(targets + 1.0)        # logGamma(Y+1)
    nb_term = is_nonzero * (                           # Eq 10
        torch.lgamma(targets + phi) - torch.lgamma(phi) - log_factorial
        + phi * (torch.log(phi + EPS) - torch.log(phi + mu + EPS))
        + targets * (torch.log(mu + EPS) - torch.log(phi + mu + EPS)) )

    return -torch.mean(zero_term + nb_term)           # Eq 11
```


7. The federated averaging (file: federated.py)

Equation 2 - the global goal.

Eq 2: minimise $J(w) = (1/|R|) * \text{sum over regions } i \text{ of } J_i(w_i)$

What it says. The whole system's goal is to make the average of all regions' losses as small as possible. Every region counts equally.

Equation 12 - how the server combines the models.

Eq 12: $W = (1/|R|) * \text{sum over clients } i \text{ of } w_i$

What it says. The new global weights W are just the average of every client's weights w_i . That is all FedAvg does. (Our code uses the standard sample-weighted average, so clients with more data count a little more.)

Why so simple? Because the ZINB loss already did the hard work on each client. The paper's whole message: get the local model and loss right, and the server can stay simple, fast, and scalable.

The code (one full round + the averaging):

```
for rnd in range(1, rounds + 1):
    weight_sets, sample_counts = [], []
    for cid in client_ids:
        local = TCN_SD(...); set_weights(local, global_w)
        strategy.local_train(local, loaders[cid], lr, local_epochs, global_w, cid)
        weight_sets.append(get_weights(local))
        sample_counts.append(len(loaders[cid].dataset))
    global_w = strategy.aggregate(weight_sets, sample_counts, ...) # FedAvg = Eq 12

def fedavg(weight_sets, sample_counts):
    total = float(sum(sample_counts))
    return [ sum(w[i]*n for w,n in zip(weight_sets, sample_counts)) / total
            for i in range(len(weight_sets[0])) ]
```

8. Every file, and what it does

File	What it contains (paper link)
data.py	Windows, time split, Head/Mid/Tail, the 4 subsets (Section 5).
model.py	The TCN-SD model = Equations 3-8.
loss.py	The sparsity-aware ZINB loss = Equations 9-11.
federated.py	FedAvg training loop (Eq 12) + centralized & separated baselines.
strategies.py	The 9 comparison methods (FedProx, SCAFFOLD, Multi-Krum...).
evaluate.py	Macro-F1, Micro-F1, per-category F1 and F2 (Section 5.4).
cities.py	LA and Chicago settings + the crime-category names.
run.py	The command you run; ties everything together and saves results.
scripts/preprocess_chicago.py	Builds the Chicago dataset from the city open-data portal.

9. How we measure success (metrics)

The task is multi-label: for each day and each of the 8 crime types the model says yes(1) or no(0). We compare predictions with the truth using F-scores.

- **Precision**: of the crimes we predicted, how many were right?
- **Recall**: of the crimes that really happened, how many did we catch?
- **F1**: a balance of precision and recall.
- **Macro-F1**: compute F1 for each crime type, then average. Rare types count as much as common ones - so this is the strict, important score for sparse data.
- **Micro-F1**: pool everything first, so common types (theft) dominate. Usually higher and easier.
- **F2**: like F1 but weights recall more. Useful for crime because missing a real crime (false negative) is worse than a false alarm.

The prediction rule in code is simply: crime is predicted when $\pi_i > 0.5$.

```
macro = f1_score(y, preds, average='macro')    # equal weight per crime type
micro = f1_score(y, preds, average='micro')    # dominated by common types
per_f2 = fbeta_score(y, preds, beta=2, average=None) # recall-focused, per type
```

10. The nine baseline methods we compare against

The paper argues that fancy *server* tricks are NOT what makes FedCrime good - the loss is. To prove it, all these methods use the SAME TCN-SD model and SAME ZINB loss, and differ only in how the server combines models (or add a small extra term on the client). If FedCrime (plain FedAvg) matches or beats them, the point is made.

Method	Idea in one line	Type
FedAvg = FedCrime	Simple average of client weights.	aggregation
FedAvgM	Average + 'momentum' to smooth updates across rounds.	aggregation
FedProx	Adds a term stopping a client drifting too far from the global model.	local term
SCAFFOLD	Uses 'control variates' to correct client drift.	local term
FedTrimAvg	Throws away the highest/lowest values before averaging.	robust agg.
Multi-Krum	Keeps only the most agreeing clients, drops outliers.	robust agg.
Bulyan	Multi-Krum selection, then a trimmed mean. Extra robust.	robust agg.
MOON	Adds a 'contrastive' term pulling client features toward the global.	local term
FBLG	Groups similar clients (by a divergence score) before averaging.	clustering

All nine live in **strategies.py** behind one simple interface, so you can run any of them with `--method <name>` or all at once with `--method all`.

11. Our results vs the research paper

These are the global-test Macro-F1 and Micro-F1 scores our code produced on the real Los Angeles data, next to the paper's published FedCrime numbers (Table 2). Higher is better; the score falls as the subset gets sparser (Alpha $\rightarrow$ Omega), exactly as the paper reports.

Subset	Ours Macro-F1	Paper Macro-F1	Ours Micro-F1	Paper Micro-F1
SET-Alpha	71.70	72.16	73.62	74.21
SET-Beta	65.18	61.70	67.18	63.98
SET-Gamma	55.81	48.54	57.65	50.42
SET-Omega	49.42	48.43	50.79	50.23

Key idea: Our numbers match the paper closely on Alpha and Omega and are a bit higher on the harder Beta/Gamma subsets. Small differences are expected because the paper does not publish the exact areas it used in each subset - we rebuild the subsets with the described rule (20/30/50 split + sampling).

What to notice (talking points):

1. Macro-F1 drops steadily as sparsity rises - this confirms the paper's core claim that zero-inflation makes learning harder.
2. Micro-F1 stays high and moves little - because it is dominated by common crimes like theft, exactly as the paper says.
3. The training loss falls and then flattens (stable convergence) within the first rounds - matching Figure 8 in the paper, where FedCrime is the smoothest, lowest curve.
4. When we ran all nine baselines on the same subset, plain FedCrime (FedAvg) stayed at the top with the lowest, most stable loss - proving the paper's argument that the loss, not the server trick, is what matters.

12. How to run it yourself

```
# one-time setup
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# quick test (5 rounds)
python run.py --city la --subset alpha --rounds 5

# full paper run, all four subsets
python run.py --city la --subset all --rounds 100

# compare every baseline against FedCrime
python run.py --city la --subset gamma --method all --rounds 100

# Chicago (build the data first, then run)
python scripts/preprocess_chicago.py --out data/chi_crime.csv
python run.py --city chicago --subset all --rounds 100
```

13. Honest limitations (good to mention)

- The exact areas the paper used in each subset are not published, so our subsets are rebuilt from the described rule; numbers are therefore close but not identical.
- Following the authors' released code, the TCN uses same-length padding and the final yes/no decision uses $p_i > 0.5$.
- Chicago data is rebuilt live from the city open-data portal; its category counts match the paper very closely (theft ~57k, battery ~49k, robbery ~10k).

End of study guide. Read sections 5, 6 and 7 most carefully - the formulas there (Eq 3-12) are the parts an examiner is most likely to ask about.